Overview

1. C Runtime System
   - C Runtime System
   - C Standard Library
   - Processes and Libraries
2. Error Handling in C
   - `errno`
   - `longjmp`
   - Signals
3. Application of `longjmp` and Signals
   - Thread Scheduling
   - Exceptions
4. Interactions with Other Runtime Systems
   - Interactions

The C Runtime System

The language C is the most common language for low-level system programming. Therefore, the runtime system of C is often an (indirect) part of other runtime systems, e.g., for high-level languages.

Indirect contact with the C runtime system is a common phenomenon, especially in Unix-like operating systems. There, it is mostly a “wrapper” around the functions of the operating system kernel. On other operating systems that are less tightly tied to C (e.g., Microsoft Windows), the C runtime environment implements its services with the help of the operating system’s functions.

C Runtime System

Because C is a rather low-level language, its runtime system does not provide advanced services. The main task is the (rather low) abstraction from the services of the operating system kernel.

Main tasks:

- memory management (`malloc`, `free`, `...`)
- process and signal management
- I/O
- error handling
- string manipulation
- threads, locks (newer standards)

Since C has no exception handling mechanism, error handling is more complex and has more pitfalls than, e.g., in Java.

Overview

1. C Runtime System
   - C Runtime System
   - C Standard Library
   - Processes and Libraries
2. Error Handling in C
   - `errno`
   - `longjmp`
   - Signals
3. Application of `longjmp` and Signals
   - Thread Scheduling
   - Exceptions
4. Interactions with Other Runtime Systems
   - Interactions

C Standard Library

In its early days (1969-1978), neither the language C nor its standard library were precisely specified.

1978 “The C Language” by B. Kernighan and D. Ritchie specifies the language C (“K&R C”).

1989 ANSI X3.159-1989 (“C 89”) specifies C and the standard library.

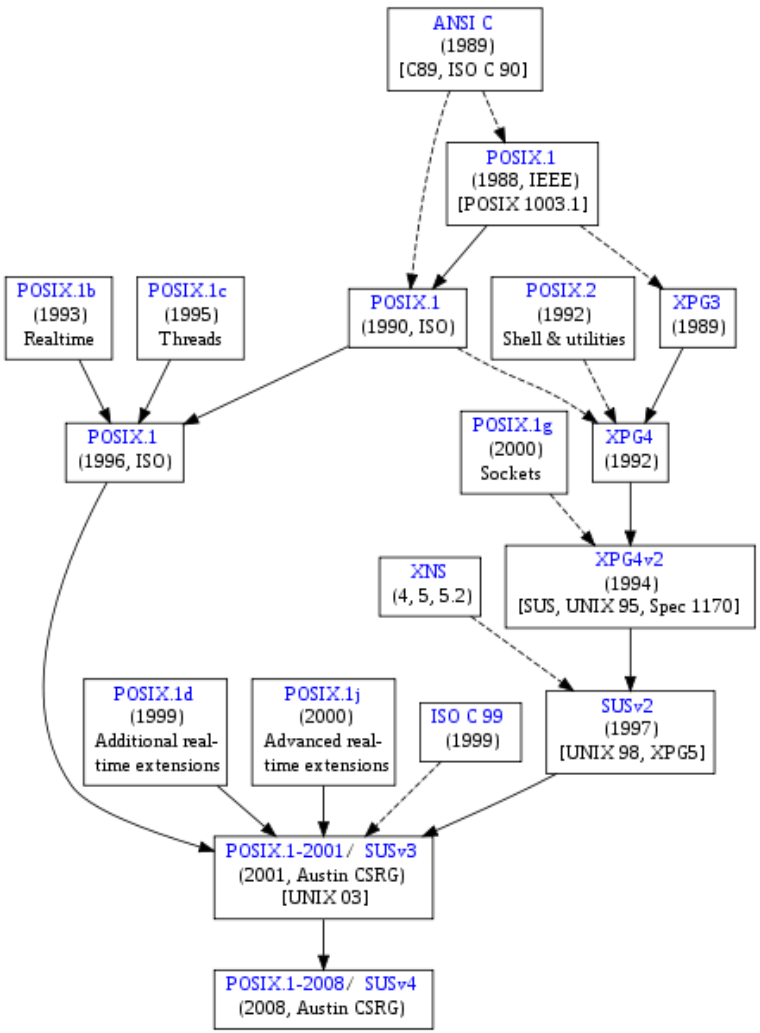
1990 ISO/IEC 9899:1990 (ANSI C with small changes)

1999 ISO/IEC 9899:1999 (“C 99”) extends the language (some similarities to Java/C++) and the standard library.

2011 ISO/IEC 9899:2011 (“C 11”) supports multi-threading, bounds checking

2018 ISO/IEC 9899:2018 (“C 17”, only small updates w.r.t. C 11)

POSIX Standard Library



- POSIX specifies library functions and command line tools.
- The POSIX standard library complements ANSI/ISO C with functions specific for Unix-like operating systems.
- MacOS X and Solaris are (completely) POSIX compliant.
- Linux and FreeBSD are nearly POSIX compliant.
- Windows: POSIX layer (“services for Unix”) for selected (older) versions; alternatives: Cygwin/MinGW; recently: Linux subsystems (WSL, WSL2).

Overview

1. C Runtime System
   - C Runtime System
   - C Standard Library
   - Processes and Libraries
2. Error Handling in C
   - `errno`
   - `longjmp`
   - Signals
3. Application of `longjmp` and Signals
   - Thread Scheduling
   - Exceptions
4. Interactions with Other Runtime Systems
   - Interactions

Processes

A process is a program that is currently executing. It consists of

- the code that is executed (“text segment”), and
- the data that is manipulated.

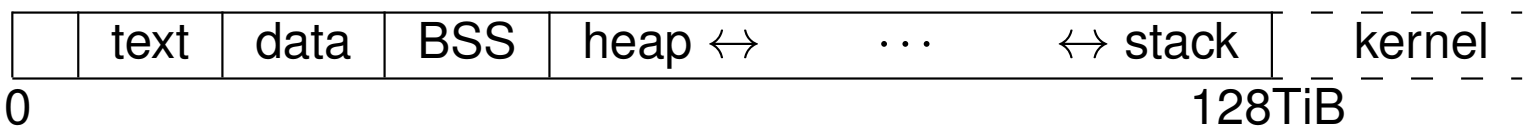
The data area of a process consists of

- the initialized data (“data segment”),
- the uninitialized data (“BSS segment”),
- the heap, and
- the stack.

Heap and stack grow and shrink (usually) as required, the initialized and uninitialized data areas (data and BSS segments) are loaded together with the program code (text segment) from the program binary.

Memory Layout

The virtual (logical) memory of a process is typically organized (approximately) as shown:



Heap and stack grow dynamically (in opposite directions). When heap and stack meet, the process has to be terminated. The sizes of heap and stack can be limited (cf., for example, the command `ulimit`).

The 128 TiB mark for the boundary between user space (program data and code) and kernel is a typical value on 64-bit Linux systems (`x86_64`). On 32-bit systems, the boundary is typically at 2 GiB ($= 2^{31}$).

Physical and Virtual Memory

The layout of a process's virtual memory is, as the name suggests, virtual, i.e., independent from the physical placement of data (and code) in memory cells.

The CPU maps logical addresses (in virtual memory) by the help of the *page tables* (which are managed by the operating system) to physical addresses. This mapping is performed in the granularity of a *page*, which is typically 4 KiB. . . 1MiB in size.

A physical memory page can be mapped to the logical address space of several processes (maybe at different virtual addresses in the respective process's address space); therefore, it is possible (even without swap memory) that the sum of the virtual address spaces on a system exceeds the size of the physical memory.

Binary Formats

The binary representation of a program (in a file on a storage medium) consists of several sections, including the text, data and BSS segments.

Common binary formats:

- PE (portable executable): Microsoft Windows (32- and 64-bit)
- ELF (executable and linking format): many systems of the Unix line, including Linux, FreeBSD, Solaris
- Mach-O: MacOS X

Binaries in ELF format can be analyzed with the tool `readelf`.

Libraries

Practically every program today needs to use software libraries (collections of functions); almost every program uses at least the C library.

Libraries come in two principal varieties:

- *static libraries* are integrated at compile time (more precisely: linking time) into the program binary,
- *dynamic libraries* (“shared objects”) are combined with the program code only at run time (when the program is started or even later).

Compilation Process

1. compilation: source code → assembly code
   file.c / file.hs / . . . → file.s
   e.g., `gcc -S -o file.s file.c`
2. assembly: assembly code → object code (machine code)
   file.s → file.o
   e.g., `gcc -c -o file.o file.s`
3. object code → binary or library
   file.o → file (binary)
   e.g., `gcc -o file file.o`

   file.o → libMYLIB.a (static library)
   e.g., `ar r libMYLIB.a file.o`

   file.o → libMYLIB.so (dynamic library)
   e.g., `gcc -shared -fPIC -o libMYLIB.so file.o`

(The frontend `gcc` calls the right tool for a given task, i.e., dependeng on the command line flags, it calls the preprocessor `cpp`, the compiler `cc1`, the assembler `as`, the linker `ld`, etc.)

How to Use a Library

To use a library, several flags (options) for the linker (here: GNU ld) are available:

`-L`*path* look for libraries in *path*,

`-l`*bib* use `lib`*bib*`.a` or `lib`*bib*`.so`.

For dynamic libraries, there is also

`-rpath` *path* to look in *path* at run time for dynamic libraries.

Example

product.c

```
int product(int a, int b) { return a*b; }
```

main.c

```
int product(int a, int b);

int main(void) {
    printf("4*5 = %d\n", product(4,5));
    return 0;
}
```

Compile, link, run

```
gcc -fPIC -shared -o libproduct.so product.c
gcc -o main main.c -L. -lproduct

./main
```

Example

product.c

```
int product(int a, int b) { return a*b; }
```

main.c

```
int product(int a, int b);

int main(void) {
    printf("4*5 = %d\n", product(4,5));
    return 0;
}
```

Compile, link, run

```
gcc -fPIC -shared -o libproduct.so product.c
gcc -o main main.c -L. -lproduct
export LD_LIBRARY_PATH=.
./main
```

Example

product.c

```
int product(int a, int b) { return a*b; }
```

main.c

```
int product(int a, int b);

int main(void) {
    printf("4*5 = %d\n", product(4,5));
    return 0;
}
```

Compile, link, run

```
gcc -fPIC -shared -o libproduct.so product.c
gcc -o main main.c -L. -lproduct -Wl,-rpath,.

./main
```

Loading Libraries During Execution

Dynamic libraries cannot only be loaded when a program is started but also at a later time. To this end, one can use the functions in libdl.

Dynamic loading of a library

```
void *dlopen(const char *libname, int flags);
```

open library `libname`; returns a handle that (when not `NULL`) identifies the library. Flags:

`RTLD_LAZY`	resolve symbols in the library (i.e., references to other libraries) only when needed,
`RTLD_NOW`	resolve symbols at once,
`RTLD_GLOBAL`	symbols in the loaded library are made available for libraries loaded later,
`RTLD_LOCAL`	symobls of the loaded library are not visible to the outside.

```
int dlclose(void *handle);
```

close a library.

Finding Symbols in a Library

Find a symbol

```
void *dlsym(void *handle, const char *symname);
```

Look up symbol `symname` in the library identified by `handle` and return its address. The address of a symbol can be `NULL`; therefore, `NULL` is *not* a reliable indicator for an error.

```
char *dlerror(void);
```

Return a description for the last error encountered by `libdl`, or `NULL` when no error has occurred.

Example

```
void *hdl = dlopen("libm.so", RTLD_LAZY | RTLD_GLOBAL);
if (hdl != NULL) {
    double (*sin_fn)(double);
    char *err;
    dlerror(); // clear previous error (if any)
    sin_fn = dlsym(hdl, "sin");
    err = dlerror();
    if (err == NULL)
        printf("sin(1.0) = %g\n", sin_fn(1.0));
    else
        fprintf(stderr, "Error looking up sin: %s", err);
    dlclose(hdl);
} else
    fprintf(stderr, "Error opening libm.so: %s", dlerror());
```

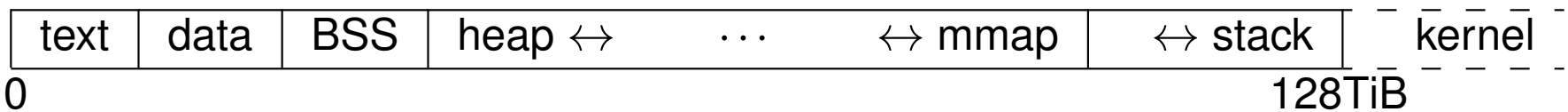
Memory Layout with Dynamic Libraries

Question: Where are text, data and BSS segments of dynamically loaded libraries put in a process's address space?

Memory Layout with Dynamic Libraries

Question: Where are text, data and BSS segments of dynamically loaded libraries put in a process's address space?

Answer: Between heap and stack there is another memory area, the so-called *mmap area*.



Dynamic libraries are placed in the mmap area, together with explicitly created memory mappings (mmap = memory mapping). A library can (because of virtual memory) be mapped into the address spaces of several processes, i.e., dynamic libraries save (physical) memory compared to static libraries (which are copied into every binary).

MMAP

A process can use the memory mapping facilities to map parts of arbitrary files into its address space. In other words, it is then possible to write to and read from a file using memory operations instead of I/O operations.

MMAP

A process can use the memory mapping facilities to map parts of arbitrary files into its address space. In other words, it is then possible to write to and read from a file using memory operations instead of I/O operations.

mmap

```
void *mmap(void *addr, size_t length, int prot,
           int flags, int fd, off_t offset);
```

Maps a part with `length` bytes starting at `offset` of the file identified by `fd`.

`addr` is only a request (wish) where to create the mapping (often simply `NULL` is provided), the actual address of the mapping is returned by `mmap`. `prot` determines the access rights, e.g., `PROT_READ`, `PROT_WRITE` for read and write access. `flags` can be, among others, `MAP_SHARED` (several processes can access the mapped area) or `MAP_PRIVATE` (no sharing, i.e., a write access leads to copying of the respective memory page, so-called “copy on write”). `offset` must be a multiple of the memory page size.

Example

```
fd = open(...);
if (fd != -1) {
    size_t length = ...;  /* Determine the length of the file */
    char *addr = mmap(NULL, length, PROT_READ, MAP_SHARED, fd, 0);
    if (addr != MAP_FAILED) {
        /* Output all characters from the file */
        for (i=0; i<length; i++) {
            printf("%c", addr[i]);
        }
        printf("\n");
        munmap(addr, length);
    }
    close(fd);
}
```

LD.SO

Loading of dynamic libraries is not performed by the operating system kernel but by the *dynamic loader* ld.so. In the section `interp` (for “interpreter”) of an ELF file, the path to the dynamic loader is hard-coded, e.g., `/lib64/ld-linux-x86-64.so.2` under `x86_64` Linux. The dynamic loader interprets the information about dynamic libraries in the program file. The dynamic loader assigns an address in memory to each dynamic library. Because the address at which a dynamic library is loaded is not fixed, dynamic libraries should be compiled as *position independent code* (compiler option `-fPIC`). Such code does not use absolute addresses for jump instructions or data access, only relative addressing is used. The load address of the library does not matter for position independent code.

When code is not position independent, the loader has to correct the addresses in data references and jump instructions, i.e., the code has to be changed. For this purpose, the binary contains so-called *relocation tables*.

Overview

1. C Runtime System
   - C Runtime System
   - C Standard Library
   - Processes and Libraries
2. Error Handling in C
   - `errno`
   - `longjmp`
   - Signals
3. Application of `longjmp` and Signals
   - Thread Scheduling
   - Exceptions
4. Interactions with Other Runtime Systems
   - Interactions

Error Handling in C with `errno`

Because C does not have exceptions, most functions of the C runtime system signal errors through their return value. Often `-1` or `NULL` is the indicator that an error has occurred. The cause of the error can be determined from the global variable `int errno`.

Example:

```
ssize_t res = read(fd, data, count);
if (res == -1) {
    fprintf(stderr, "Error number %d occurred.\n, errno);
}
```

Error Handling in C with `errno`

Because C does not have exceptions, most functions of the C runtime system signal errors through their return value. Often `-1` or `NULL` is the indicator that an error has occurred. The cause of the error can be determined from the global variable `int errno`.

Example:

```
ssize_t res = read(fd, data, count);
if (res == -1) {
    fprintf(stderr, "Error number %d occurred.\n, errno);
}
```

A textual description of an error can be obtained with `char~*strerror(int errnum)` or one uses one of the functions provided to output error messages, e.g.:

- `void perror(const char *s)`
- `void error(int status, int errnum, const char *fmt, ...);`

Using `errno`

Sometimes one can see the following code pattern for error handling:

```
errno = 0;
read(fd, ...);
write(fd, ...);
... /* more function calls that may set errno */
if (errno != 0) {
   /* some call has failed, handle errors */
   ...
}
```

Using `errno`

Sometimes one can see the following code pattern for error handling:

```
errno = 0;
read(fd, ...);
write(fd, ...);
... /* more function calls that may set errno */
if (errno != 0) {
   /* some call has failed, handle errors */
   ...
}
```

This use of `errno` is *not correct*. `errno` can be set to a value other than 0 even when a function call succeeds. Therefore, almost always the return value has to be checked before `errno` is inspected.

Using `errno`

Sometimes one can see the following code pattern for error handling:

```
errno = 0;
read(fd, ...);
write(fd, ...);
... /* more function calls that may set errno */
if (errno != 0) {
   /* some call has failed, handle errors */
   ...
}
```

This use of `errno` is *not correct*. `errno` can be set to a value other than 0 even when a function call succeeds. Therefore, almost always the return value has to be checked before `errno` is inspected.

There are a few exceptions, e.g., the function `getpriority` yields -1 as error indicator *or* when the scheduling priority is actually -1. Here, one has to set `errno=0` before the call and when -1 is returned, `errno` has to be checked.

Overview

1. C Runtime System
   - C Runtime System
   - C Standard Library
   - Processes and Libraries
2. Error Handling in C
   - errno
   - longjmp
   - Signals
3. Application of longjmp and Signals
   - Thread Scheduling
   - Exceptions
4. Interactions with Other Runtime Systems
   - Interactions

`longjmp` – non-local goto

The big advantace of exceptions is (in addition to typing of error classes) the possibility to perform controlled non-local jumps in the control flow of the program.

longjmp – non-local goto

The big advantace of exceptions is (in addition to typing of error classes) the possibility to perform controlled non-local jumps in the control flow of the program.

The C runtime system also offers non-local jumps but these are neither typed nor controlled. Using

```
int setjmp(jmp_buf env);
```

the execution context can be saved to `env` and when needed, it can be restored with

```
int longjmp(jmp_buf env, int val);
```

The program execution then returns to the `setjmp` call. `setjmp` returns 0 when `env` is initialized and `val` when it is reached again through a call to `longjmp`.

The execution context contains (mostly) the processor registers but not the program memory.

Example for `longjmp`

```
jmp_buf env;

void f(int arg) {
    ...
    if (some_error)
        longjmp(env, 1);
    ...
}

int main(void) {
    ...
    if (setjmp(env) == 0) {
        for (int i=0; i<10; i++)
            f(i);
    } else
        printf("Oops, f() failed\n");
    ...
}
```

How `longjmp` Works

Because `longjmp` resets the registers of the processor to the values the registers had when `setjmp` was called, the instruction pointer (RIP on `x86_64`) and the registers for stack management (RBP and RSP on `x86_64`) are reset in particular.

How `longjmp` Works

Because `longjmp` resets the registers of the processor to the values the registers had when `setjmp` was called, the instruction pointer (RIP on `x86_64`) and the registers for stack management (RBP and RSP on `x86_64`) are reset in particular.

Therefore, `longjmp` lets the execution flow jump back to the call to `setjmp` and the stack has the same layout it had there. The stack records of functions that have been called between `setjmp` and `longjmp` are removed from the stack.

How `longjmp` Works

Because `longjmp` resets the registers of the processor to the values the registers had when `setjmp` was called, the instruction pointer (RIP on `x86_64`) and the registers for stack management (RBP and RSP on `x86_64`) are reset in particular.

Therefore, `longjmp` lets the execution flow jump back to the call to `setjmp` and the stack has the same layout it had there. The stack records of functions that have been called between `setjmp` and `longjmp` are removed from the stack.

Note: The stack frame in which `setjmp` has been called must still exist when `longjmp` is used, i.e., the function that called `setjmp` must not have ended.

Local Variables and `longjmp`

What value does the following program print for `i`?

```
int main(void) {
    jmp_buf env;
    int i = 42;
    if (setjmp(env) == 0) {
        i = 23;
        longjmp(env, 1);
    } else {
        printf("i = %d\n", i);
    }
}
```

Local Variables and `longjmp`

What value does the following program print for `i`?

```
int main(void) {
    jmp_buf env;
    int i = 42;
    if (setjmp(env) == 0) {
        i = 23;
        longjmp(env, 1);
    } else {
        printf("i = %d\n", i);
    }
}
```

Both 23 and 42 are possible. Why?

Local Variables and `longjmp`

What value does the following program print for `i`?

```
int main(void) {
    jmp_buf env;
    int i = 42;
    if (setjmp(env) == 0) {
        i = 23;
        longjmp(env, 1);
    } else {
        printf("i = %d\n", i);
    }
}
```

Both 23 and 42 are possible. Why? When the compiler puts `i` in a processor register, its value is reset to 42 by `longjmp`. But when `i` is stored on the stack, the respective memory cell still has the value 23 after `longjmp`. ⇝ Local variables that are modified between `setjmp` and `longjmp` must be declared `volatile` for they stay in memory all the time (and the compiler does not put them into registers).

Overview

1. C Runtime System
   - C Runtime System
   - C Standard Library
   - Processes and Libraries
2. Error Handling in C
   - errno
   - longjmp
   - Signals
3. Application of longjmp and Signals
   - Thread Scheduling
   - Exceptions
4. Interactions with Other Runtime Systems
   - Interactions

Signals

The `longjmp` mechanism is for an explicitly requested non-local change in control flow. For implicit (i.e., not requested explicitly by the program) exceptions, there is the *signal* mechanism.

A signal is a message to a process that a certain event has occurred. Examples:

- illegal memory access/segmentation fault (`SIGSEGV`), bus error (`SIGBUS`)
- illegal instruction (`SIGILL`)
- arithmetic error (`SIGFPE`)
- keyboard interrupt with Ctrl+C (`SIGINT`)
- connection loss to the terminal (`SIGHUP`)
- program termination with “term” or “kill” (`SIGTERM`, `SIGKILL`)
- timer interrupt (`SIGALRM`)
- exit of a child process (`SIGCHLD`)
- broken pipe (`SIGPIPE`)

Signal Handlers

Most signals cause one of three possible responses:

- The signal is ignored.
- The signal causes program termination (sometimes with a core dump).
- A user-defined handler for the signal is called.

The default for most signals is to cause program termination. Only `SIGCHLD` is ignored by default. Stop and continue signals case the program execution to halt or resume, respectively (e.g., some shells send a stop signal to the foreground process when Ctrl+Z is pressed).

Signals can be blocked temporarily, i.e., their handling is delayed until they are unblocked again $\rightsquigarrow$ function `sigprocmask`.

For all the details, see `man 7 signal`.

Installation of Signal Handlers

Traditionally, signal handles have been installed with the function `signal` but this function has different semantics on different systems. Better alternative:

```
struct sigaction {
    void     (*sa_handler)(int);
    void     (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t   sa_mask;
    int        sa_flags;
};

int sigaction(int signum, const struct sigaction *act,
              struct sigaction *oldact)
```

`sigaction` installs a signal handler described by `act` for the signal `signum` with handler function `sa_sigaction`, when `SA_SIGINFO` is set in `sa_flags`, and `sa_handler` otherwise. `sa_mask` tells which signals shall be blocked (in addition to `signum`) while the handler is executed. To manipulate signal sets (such as `sa_mask`), there are the functions `sigemptyset`, `sigaddset`, etc.

Example for `sigaction`

```c
void intr_handler(int sig) {
    printf("Interrupt received, exiting.\n");
    exit(0);
}

int main(void) {
    struct sigaction sa;
    sa.sa_handler = intr_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, NULL);
    while(1) { }

    return 0;
}
```

Signals and `longjmp`

When a signal handler exits normally (by returning from the handler function), the control flow returns to the place where the execution was interrupted by the signal. This makes sense for, e.g., timer signals.

For signals that inform the program about an error, e.g., `SIGSEGV`, this behavior ist not desirable (and does not make sense). Signal handlers can use `longjmp` to handle errors such als `SIGSEGV` oder `SIGFPE` by redirecting control flow to an error handling routine after which normal program operation can resume.

Pay attention to the signal mask: the handled signal (and possibly others) are blocked while the handler is executed. When the handler returns normally, the signal(s) are unblocked automatically. But when the handler uses `longjmp`, the signal mask has to be modified explicitly, e.g., by using `sigprocmask` oder `siglongjmp`.

sigsetjmp and siglongjmp

POSIX does not specify whether longjmp resets the signal mask to its state at the time setjmp had been called or not. The variant sigsetjmp provides a parameter savesigs to tell whether the signal mask shall be restored or not.

```
int sigsetjmp(sigjmp_buf env, int savesigs);
int siglongjmp(sigjmp_buf env, int val);
```

With savesigs=0, the signal mask is not changed by siglongjmp, with savesigs≠0 the signal mask is restored to the state it had at sigsetjmp.

Reentrancy, Thread Safetry, Interrupts

Due to asynchronous signals (e.g., timers) it can happen that a function is entered multiple times in an overlapped fashion. Example:

1. The main programm calls a function, say `malloc`.
2. A signal arrives.
3. Since the signal is not blocked, the signal handler is called.
4. The signal handler also calls `malloc`.

Is this allowed?

Reentrancy, Thread Safetry, Interrupts

Due to asynchronous signals (e.g., timers) it can happen that a function is entered multiple times in an overlapped fashion. Example:

1. The main programm calls a function, say `malloc`.
2. A signal arrives.
3. Since the signal is not blocked, the signal handler is called.
4. The signal handler also calls `malloc`.

Is this allowed? POSIX does not guarantee that this works. Reason: `malloc` manipulates global data (the data structures for heap management) and that is why the behavior is undefined.

Signal handlers may only use a restricted set of functions, namely the functions that are *reentrant*.

Reentrancy, Thread Safety

Reentrancy

A function is *reentrant* when it can be entered several times by the same thread. This implies that it

- cannot access global modifiable data (except by atomic operations),
- cannot modify its own code,
- cannot call functions which are not reentrant.

Reentrancy, Thread Safety

Reentrancy

A function is *reentrant* when it can be entered several times by the same thread. This implies that it

- cannot access global modifiable data (except by atomic operations),
- cannot modify its own code,
- cannot call functions which are not reentrant.

Reentrancy is different from thread safety.

Thread Safety

A function is *thread-safe* when it can be called at the same time by different threads.

Reentrant functions are thread-safe (when resources such as files are handled like global data). Thread-safe fuctions are *not* automatically reentrant.

Interruption of System Functions

Some functions of the runtime system (those that call kernel functions) are interrupted when a signal arrives while they are executing. They exit with an error state and set `errno` to `EINTR` (interrupted system call).

Interruption of System Functions

Some functions of the runtime system (those that call kernel functions) are interrupted when a signal arrives while they are executing. They exit with an error state and set `errno` to `EINTR` (interrupted system call).

Such functions must (should) be called again when `EINTR` is reported.

```
ssize_t res;
do {
    res = read(fd, buffer, count);
} while (res == -1 && errno == EINTR);
```

Interruption of System Functions

Some functions of the runtime system (those that call kernel functions) are interrupted when a signal arrives while they are executing. They exit with an error state and set `errno` to `EINTR` (interrupted system call).

Such functions must (should) be called again when `EINTR` is reported.

```
ssize_t res;
do {
    res = read(fd, buffer, count);
} while (res == -1 && errno == EINTR);
```

When a handler for a signal is installed with the flag `SA_RESTART`, *some* functions are restarted automatically when interrupted by the signal, see `man 7 signal` for details.

Overview

1. C Runtime System
   - C Runtime System
   - C Standard Library
   - Processes and Libraries
2. Error Handling in C
   - `errno`
   - `longjmp`
   - Signals
3. Application of `longjmp` and Signals
   - Thread Scheduling
   - Exceptions
4. Interactions with Other Runtime Systems
   - Interactions

Thread Scheduling with `longjmp` and Signals

`longjmp` allows non-local jumps. This makes it possible to emulate threads (and co-routines) by jumping between the threads using `longjmp`. The challenge lies in providing each thread with its own stack area.

Thread Scheduling with `longjmp` and Signals

`longjmp` allows non-local jumps. This makes it possible to emulate threads (and co-routines) by jumping between the threads using `longjmp`. The challenge lies in providing each thread with its own stack area.

The function `void *alloca(size_t size)` can be used to allocate space dynamically on the stack. The allocation is valid till the end of the current function. (Note: `alloca` is not standardized and its implementation is compiler-dependent.)

Our threads get separate stacks by having the stack pointer point to a different area of the stack (using `alloca`) when creating the threads.

In the following, threads are created with `new_thread` and by calling `schedule` execution switches between the threads. In the beginning, some setup/initialization in the function `init` is necessary. (Details in the tutorial!)

```
int main(void) {
    init(thread0);
}

volatile int curr, numThreads;
jmp_buf newThread, jmpbufs[N];
void (*newThreadFn)(void);

void new_thread(void (*tFn)(void)) {
    newThreadFn = tFn;
    if (setjmp(jmbpufs[curr]) == 0)
        longjmp(newThread, 1);
}

void schedule(void) {
    /* save current thread to jmpbufs
     * and activate next thread */
}

void thread0(void) {
    ...
    new_thread(thread1);
    ...
    schedule();
    ...
}
```

```
void init(void (*threadFn)(void)) {
   curr = 0; numThreads = 1;
   if (setjmp(newThread)  == 0)
       threadFn();
   else {
      curr = numThreads++;
      alloca(65536*curr);
       newThreadFn();
   }
   /* remove current thread from
    * jmpbufs and activate one of
    * the remaining threads if the
    * current thread is not the
    * last thread. */

}

void thread1(void)  {
    ...
     schedule();
     ...
     schedule();
     ...
}
```

```c
int main(void) {
    init(thread0);
}

volatile int curr, numThreads;
jmp_buf newThread, jmpbufs[N];
void (*newThreadFn)(void);

void new_thread(void (*tFn)(void)) {
    newThreadFn = tFn;
    if (setjmp(jmbpufs[curr]) == 0)
        longjmp(newThread, 1);
}

void schedule(void) {
    /* save current thread to jmpbufs
     * and activate next thread */
}

void thread0(void) {
    ...
    new_thread(thread1);
    ...
    schedule();
    ...
}
```

```c
void init(void (*threadFn)(void)) {
   curr = 0; numThreads = 1;
   if (setjmp(newThread)  == 0)
       threadFn();
   else {
      curr = numThreads++;
      alloca(65536*curr);
       newThreadFn();
   }
   /* remove current thread from
    * jmpbufs and activate one of
    * the remaining threads if the
    * current thread is not the
    * last thread. */
}

void thread1(void)  {
    ...
     schedule();
     ...
     schedule();
     ...
}
```

```
int main(void) {
    init(thread0);
}

volatile int curr, numThreads;
jmp_buf newThread, jmpbufs[N];
void (*newThreadFn)(void);

void new_thread(void (*tFn)(void)) {
    newThreadFn = tFn;
    if (setjmp(jmbpufs[curr]) == 0)
        longjmp(newThread, 1);
}

void schedule(void) {
    /* save current thread to jmpbufs
     * and activate next thread */
}

void thread0(void) {
    ...
    new_thread(thread1);
    ...
    schedule();
    ...
}
```

```
void init(void (*threadFn)(void)) {
   curr = 0; numThreads = 1;
   if (setjmp(newThread)  == 0)
       threadFn();
   else {
      curr = numThreads++;
      alloca(65536*curr);
       newThreadFn();
   }
   /* remove current thread from
    * jmpbufs and activate one of
    * the remaining threads if the
    * current thread is not the
    * last thread. */
}

void thread1(void)  {
    ...
     schedule();
     ...
     schedule();
     ...
}
```

```c
int main(void) {
    init(thread0);
}

volatile int curr, numThreads;
jmp_buf newThread, jmpbufs[N];
void (*newThreadFn)(void);

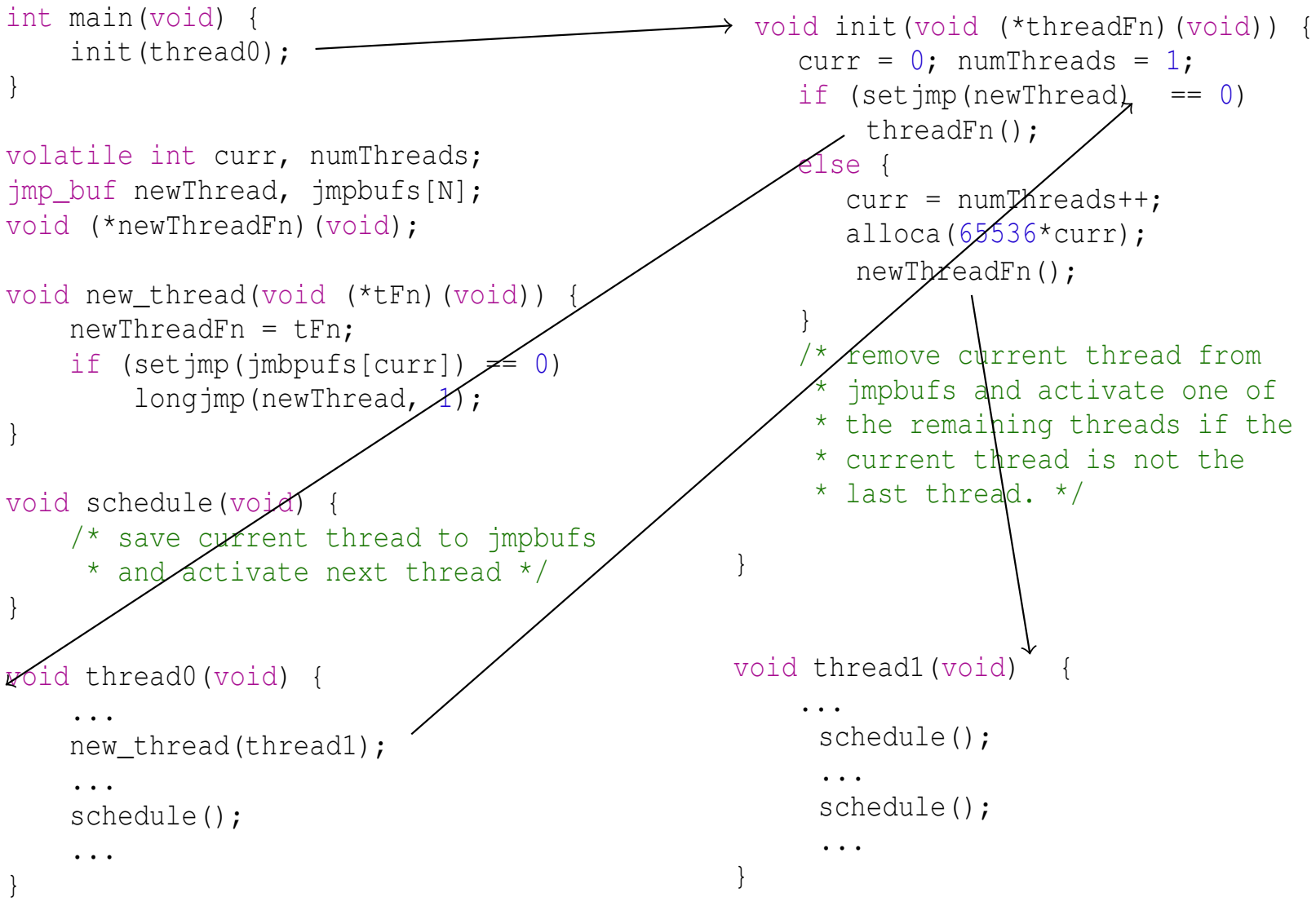
void new_thread(void (*tFn)(void)) {
    newThreadFn = tFn;
    if (setjmp(jmbpufs[curr]) == 0)
        longjmp(newThread, 1);
}

void schedule(void) {
    /* save current thread to jmpbufs
     * and activate next thread */
}

void thread0(void) {
    ...
    new_thread(thread1);
    ...
    schedule();
    ...
}
```

```c
void init(void (*threadFn)(void)) {
   curr = 0; numThreads = 1;
   if (setjmp(newThread) == 0)
      threadFn();
   else {
      curr = numThreads++;
      alloca(65536*curr);
      newThreadFn();
   }
   /* remove current thread from
    * jmpbufs and activate one of
    * the remaining threads if the
    * current thread is not the
    * last thread. */
}

void thread1(void) {
    ...
     schedule();
     ...
     schedule();
     ...
}
```



```c
int main(void) {
    init(thread0);
}

volatile int curr, numThreads;
jmp_buf newThread, jmpbufs[N];
void (*newThreadFn)(void);

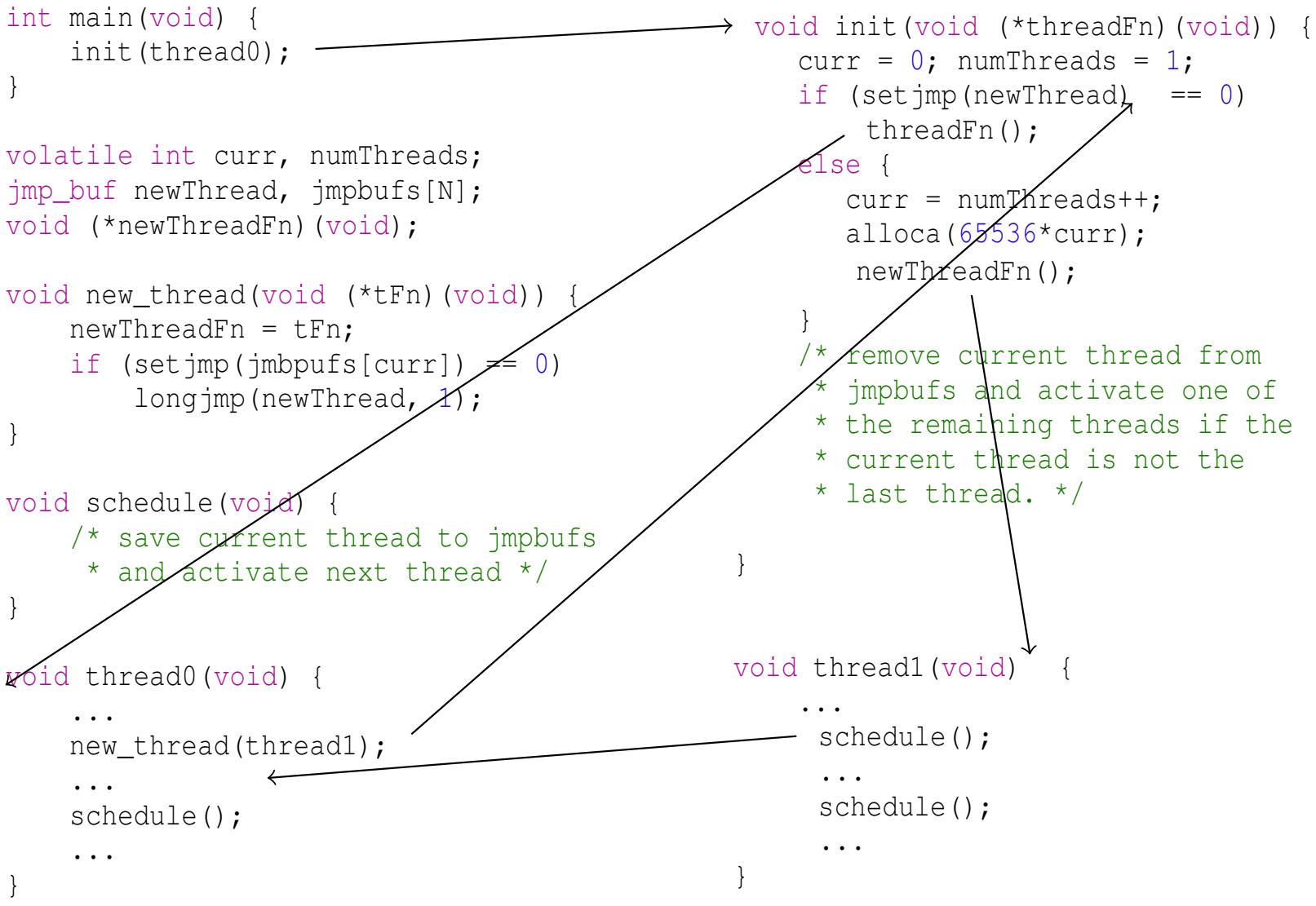
void new_thread(void (*tFn)(void)) {
    newThreadFn = tFn;
    if (setjmp(jmbpufs[curr]) == 0)
        longjmp(newThread, 1);
}

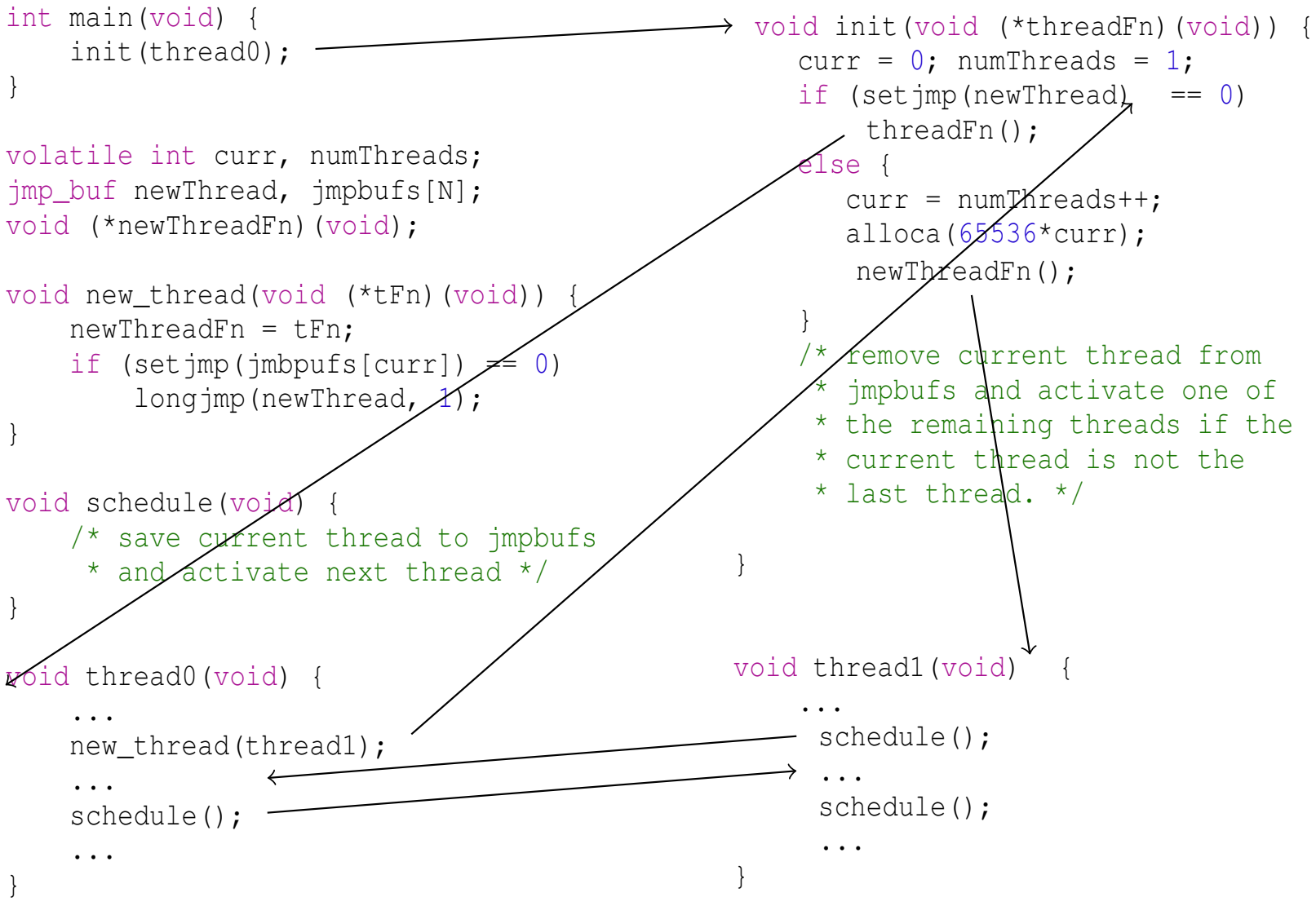
void schedule(void) {
    /* save current thread to jmpbufs
     * and activate next thread */
}

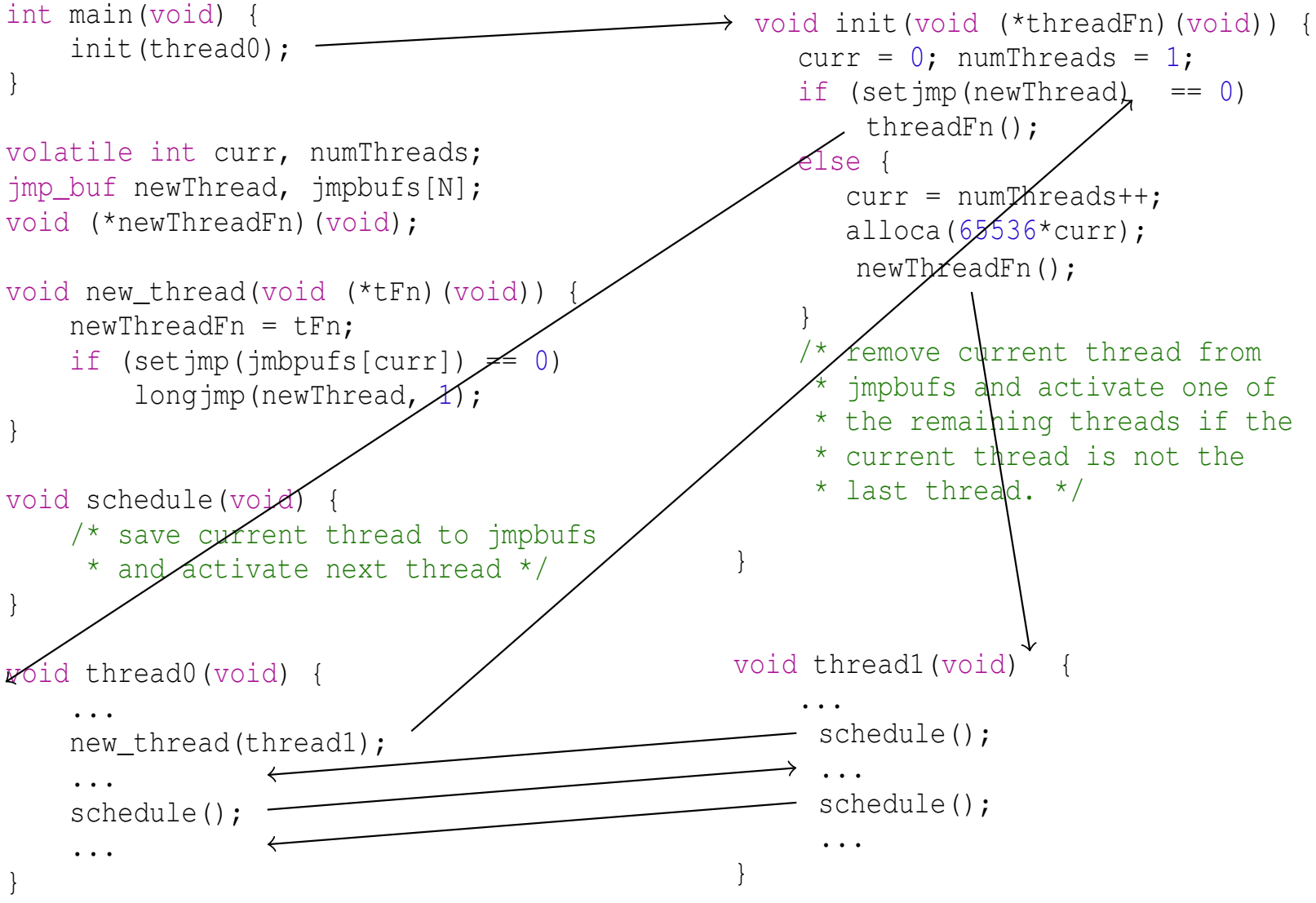
void thread0(void) {
    ...
    new_thread(thread1);
    ...
    schedule();
    ...
}
```

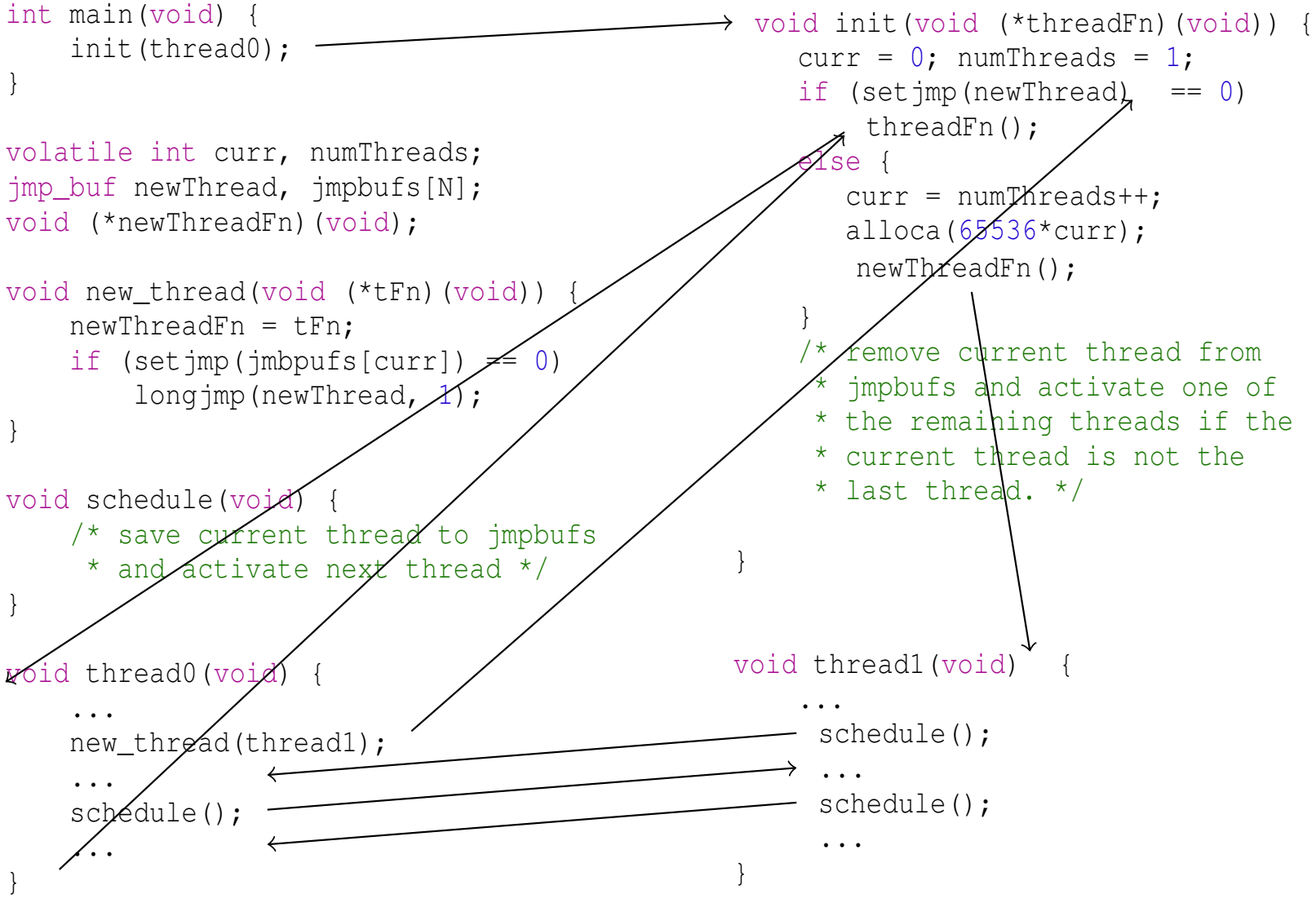
```c
void init(void (*threadFn)(void)) {
   curr = 0; numThreads = 1;
   if (setjmp(newThread) == 0)
      threadFn();
   else {
      curr = numThreads++;
      alloca(65536*curr);
      newThreadFn();
   }
   /* remove current thread from
    * jmpbufs and activate one of
    * the remaining threads if the
    * current thread is not the
    * last thread. */
}

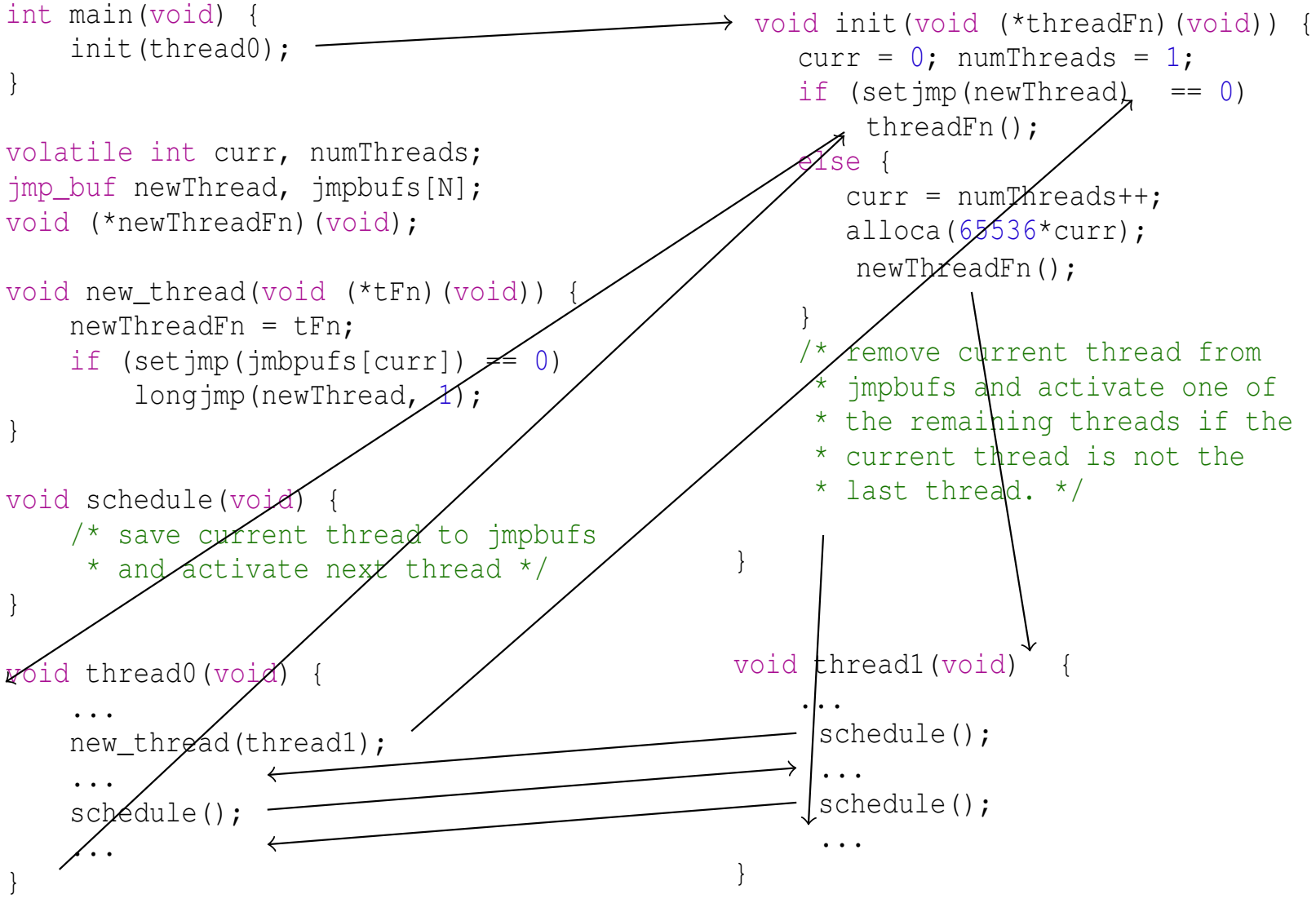
void thread1(void) {
    ...
     schedule();
     ...
     schedule();
     ...
}
```

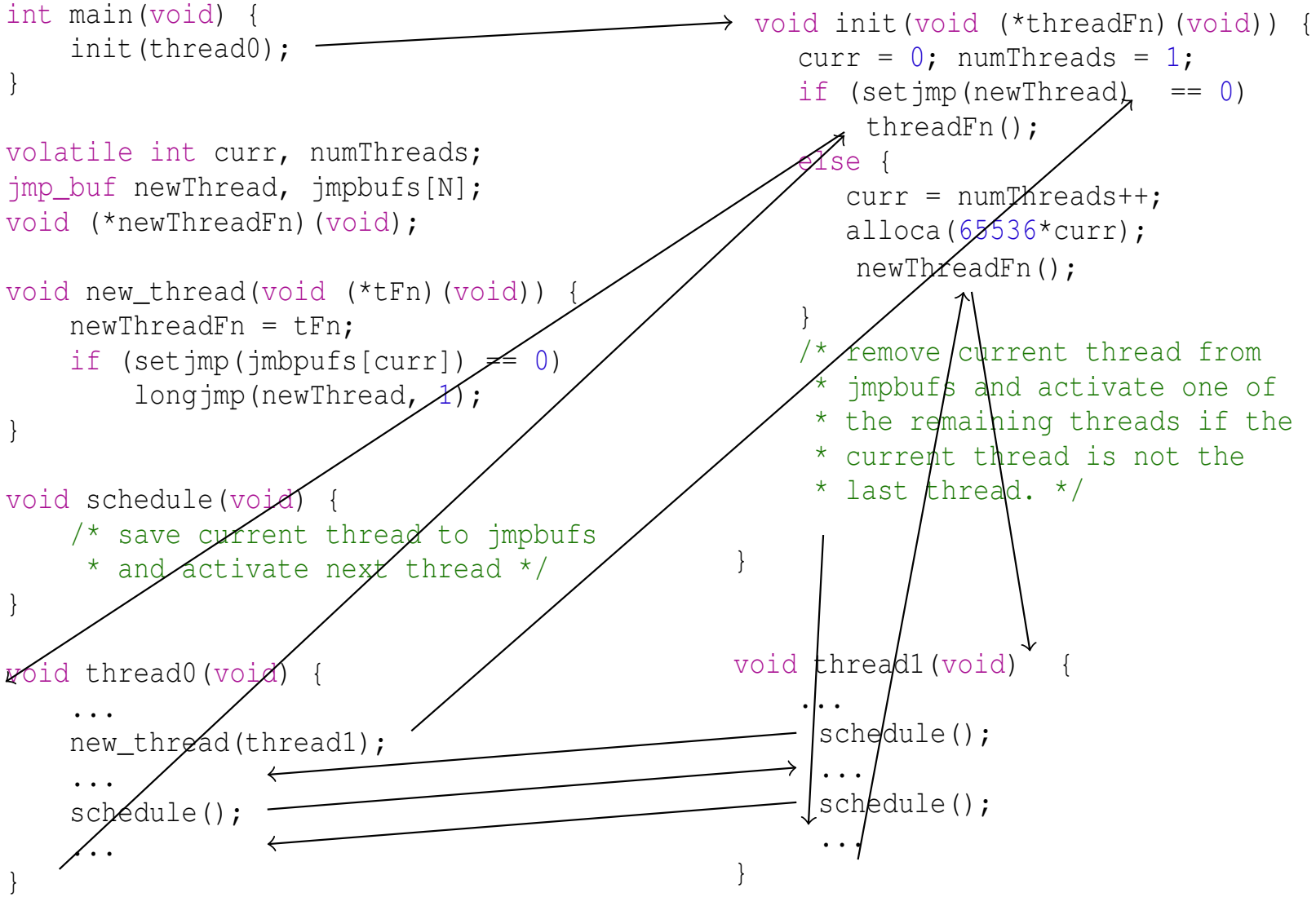

int main(void) {
init(thread0);
}
volatile int curr, numThreads;
jmp_buf newThread, jmpbufs[N];
void (*newThreadFn)(void);
void new_thread(void (*tFn)(void)) {
newThreadFn = tFn;
if (setjmp(jmbpufs[curr]) == 0)
longjmp(newThread, 1);
}
void schedule(void) {
/* save current thread to jmpbufs
* and activate next thread */
}
void thread0(void) {
...
new_thread(thread1);
...
schedule();
...
}
void init(void (*threadFn)(void)) {
curr = 0; numThreads = 1;
if (setjmp(newThread) == 0)
threadFn();
else {
curr = numThreads++;
alloca(65536*curr);
newThreadFn();
}
/* remove current thread from
* jmpbufs and activate one of
* the remaining threads if the
* current thread is not the
* last thread. */
}
void thread1(void) {
...
schedule();
...
schedule();
...
}


int main(void) {
init(thread0);
}
volatile int curr, numThreads;
jmp_buf newThread, jmpbufs[N];
void (*newThreadFn)(void);
void new_thread(void (*tFn)(void)) {
newThreadFn = tFn;
if (setjmp(jmbpufs[curr]) == 0)
longjmp(newThread, 1);
}
void schedule(void) {
/* save current thread to jmpbufs
* and activate next thread */
}
void thread0(void) {
...
new_thread(thread1);
...
schedule();
...
}
void init(void (*threadFn)(void)) {
curr = 0; numThreads = 1;
if (setjmp(newThread) == 0)
threadFn();
else {
curr = numThreads++;
alloca(65536*curr);
newThreadFn();
}
/* remove current thread from
* jmpbufs and activate one of
* the remaining threads if the
* current thread is not the
* last thread. */
}
void thread1(void) {
...
schedule();
...
schedule();
...
}


int main(void) {
init(thread0);
}
volatile int curr, numThreads;
jmp_buf newThread, jmpbufs[N];
void (*newThreadFn)(void);
void new_thread(void (*tFn)(void)) {
newThreadFn = tFn;
if (setjmp(jmbpufs[curr]) == 0)
longjmp(newThread, 1);
}
void schedule(void) {
/* save current thread to jmpbufs
* and activate next thread */
}
void thread0(void) {
...
new_thread(thread1);
...
schedule();
...
}
void init(void (*threadFn)(void)) {
curr = 0; numThreads = 1;
if (setjmp(newThread) == 0)
threadFn();
else {
curr = numThreads++;
alloca(65536*curr);
newThreadFn();
}
/* remove current thread from
* jmpbufs and activate one of
* the remaining threads if the
* current thread is not the
* last thread. */
}
void thread1(void) {
...
schedule();
...
schedule();
...
}


int main(void) {
init(thread0);
}
volatile int curr, numThreads;
jmp_buf newThread, jmpbufs[N];
void (*newThreadFn)(void);
void new_thread(void (*tFn)(void)) {
newThreadFn = tFn;
if (setjmp(jmbpufs[curr]) == 0)
longjmp(newThread, 1);
}
void schedule(void) {
/* save current thread to jmpbufs
* and activate next thread */
}
void thread0(void) {
...
new_thread(thread1);
...
schedule();
...
}
void init(void (*threadFn)(void)) {
curr = 0; numThreads = 1;
if (setjmp(newThread) == 0)
threadFn();
else {
curr = numThreads++;
alloca(65536*curr);
newThreadFn();
}
/* remove current thread from
* jmpbufs and activate one of
* the remaining threads if the
* current thread is not the
* last thread. */
}
void thread1(void) {
...
schedule();
...
schedule();
...
}


int main(void) {
init(thread0);
}
volatile int curr, numThreads;
jmp_buf newThread, jmpbufs[N];
void (*newThreadFn)(void);
void new_thread(void (*tFn)(void)) {
newThreadFn = tFn;
if (setjmp(jmbpufs[curr]) == 0)
longjmp(newThread, 1);
}
void schedule(void) {
/* save current thread to jmpbufs
* and activate next thread */
}
void thread0(void) {
...
new_thread(thread1);
...
schedule();
...
}
void init(void (*threadFn)(void)) {
curr = 0; numThreads = 1;
if (setjmp(newThread) == 0)
threadFn();
else {
curr = numThreads++;
alloca(65536*curr);
newThreadFn();
}
/* remove current thread from
* jmpbufs and activate one of
* the remaining threads if the
* current thread is not the
* last thread. */
}
void thread1(void) {
...
schedule();
...
schedule();
...
}


int main(void) {
init(thread0);
}
volatile int curr, numThreads;
jmp_buf newThread, jmpbufs[N];
void (*newThreadFn)(void);
void new_thread(void (*tFn)(void)) {
newThreadFn = tFn;
if (setjmp(jmbpufs[curr]) == 0)
longjmp(newThread, 1);
}
void schedule(void) {
/* save current thread to jmpbufs
* and activate next thread */
}
void thread0(void) {
...
new_thread(thread1);
...
schedule();
...
}
void init(void (*threadFn)(void)) {
curr = 0; numThreads = 1;
if (setjmp(newThread) == 0)
threadFn();
else {
curr = numThreads++;
alloca(65536*curr);
newThreadFn();
}
/* remove current thread from
* jmpbufs and activate one of
* the remaining threads if the
* current thread is not the
* last thread. */
}
void thread1(void) {
...
schedule();
...
schedule();
...
}

Timer for Automatic Scheduling

The call to `schedule()` in the code shown can be automated in a time-based fashion by calling `schedule()` in a signal handler for a timer event.

Using `setitimer` (see `man 2 setitimer`) one can request a periodic trigger of the timer signal. (Details in the tutorial.)

When signals are used, one should use `sigsetjmp` and `siglongjmp` to controll blocking and unblocking of the timer signal after the jumps.

Overview

1. C Runtime System
   - C Runtime System
   - C Standard Library
   - Processes and Libraries
2. Error Handling in C
   - `errno`
   - `longjmp`
   - Signals
3. Application of `longjmp` and Signals
   - Thread Scheduling
   - Exceptions
4. Interactions with Other Runtime Systems
   - Interactions

Another Application of `longjmp`: Exceptions

Many higher languages offer exceptions as a mechanism for error handling. These must be implemented efficiently by the runtime system or the virtual machine, respectively.

“Efficiently” means that code which does not cause exceptions should not be slowed down by tests for possible exceptions.

For example: should every division operation need to verify explicitly that the divisor is not 0? Should every pointer dereference require an explicit check that the pointer is not `NULL`?

Another Application of `longjmp`: Exceptions

Many higher languages offer exceptions as a mechanism for error handling. These must be implemented efficiently by the runtime system or the virtual machine, respectively.

"Efficiently" means that code which does not cause exceptions should not be slowed down by tests for possible exceptions.

For example: should every division operation need to verify explicitly that the divisor is not 0? Should every pointer dereference require an explicit check that the pointer is not `NULL`?

Optimization techniques from compiler construction can help here. For example, when a pointer variable is not changed, it is necessary to check for non-`NULL` at the first dereference only.

For `NULL` pointers and division by zero there are *no* tests necessary in the program code.

Finding the Exception Handler

Dereferences of `NULL` pointers, division by zero, etc. can be detected by hardware and the kernel/runtime system can delegate a corresponding signal to the process (`SIGSEGV`, `SIGFPE`, etc.). This way, explicit tests in the code can be avoided.

But: how can one find the exception handler (e.g., the responsible **`catch`** block in Java) and jump to it?

Finding the Exception Handler

Dereferences of `NULL` pointers, division by zero, etc. can be detected by hardware and the kernel/runtime system can delegate a corresponding signal to the process (`SIGSEGV`, `SIGFPE`, etc.). This way, explicit tests in the code can be avoided.

But: how can one find the exception handler (e.g., the responsible **`catch`** block in Java) and jump to it?

The jump can be performed using `longjmp` (or a similar mechanism). To find the exception handler, there are two approaches. The responsible exception handler is the one that

- has been installed last for the type of exception thrown $\rightsquigarrow$ dynamic solution,
- is determined by the address of the instruction that caused an error and by the type of exception $\rightsquigarrow$ static solution.

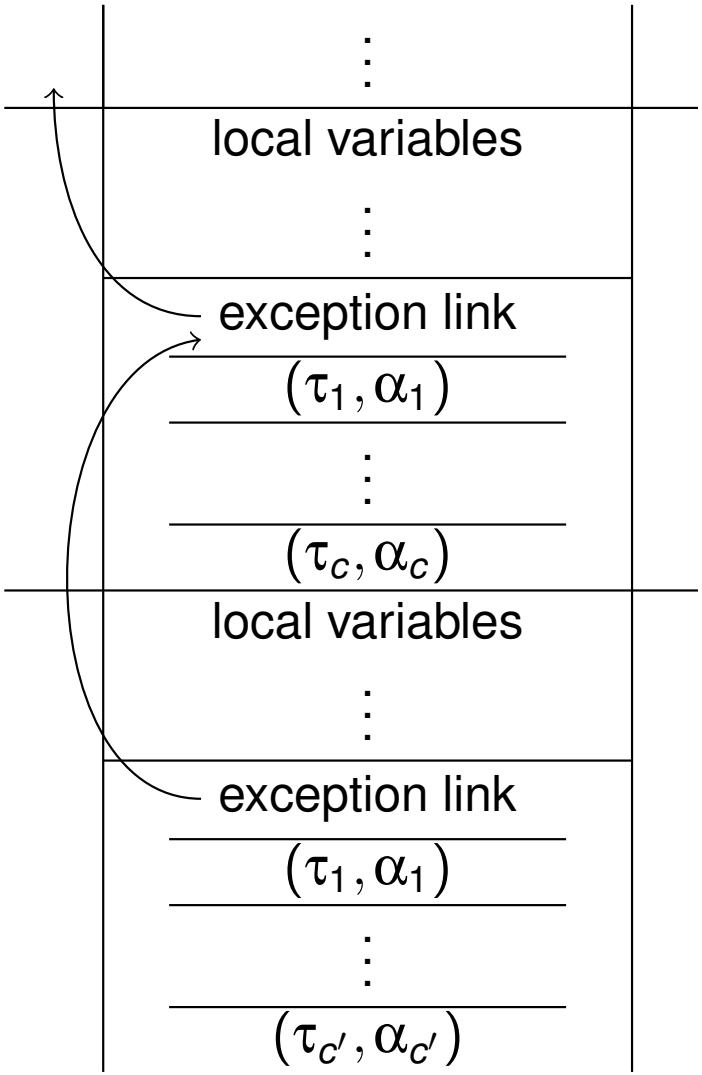
Dynamic Stack of Exception Handlers

Because the nesting of `try` blocks forms a stack at run time (the last `try` block entered is the first to be exited), one can find the exception handler the following way:

- When entering a `try` block t, for every `catch` block c_i a new pair (τ_i, α_i) is pushed on the stack of exception handlers.
  τ_i: type of the exception handled by c_i, α_i: address of the handler c_i.
- When exiting a `try` block, the elements belonging to the respective `try` block are taken from the stack.
- When an exception occurs, the stack of exception handlers is searched (starting at the top of stack) for a handler $h = (\tau, \alpha)$ with matching type τ. Let t be the `try` block belonging to h. All handlers between the top of stack and the handlers of t are removed from the stack and execution continues at address α. (Stack frames from the runtime stack have to be removed accordingly.)

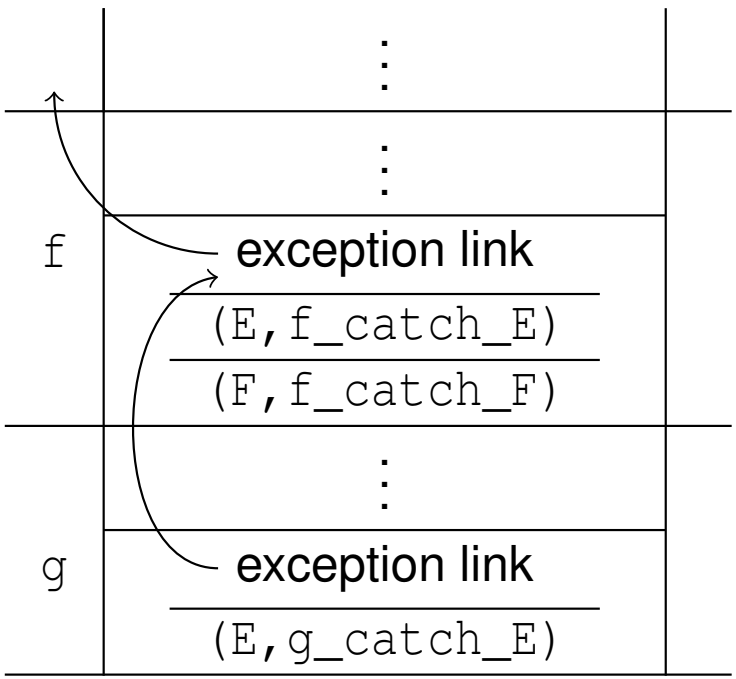
The dynamic stack of exception handlers grows and shrinks with `try` blocks $\rightarrow$ integration into the runtime stack.

Stack Frame with Exception Handlers



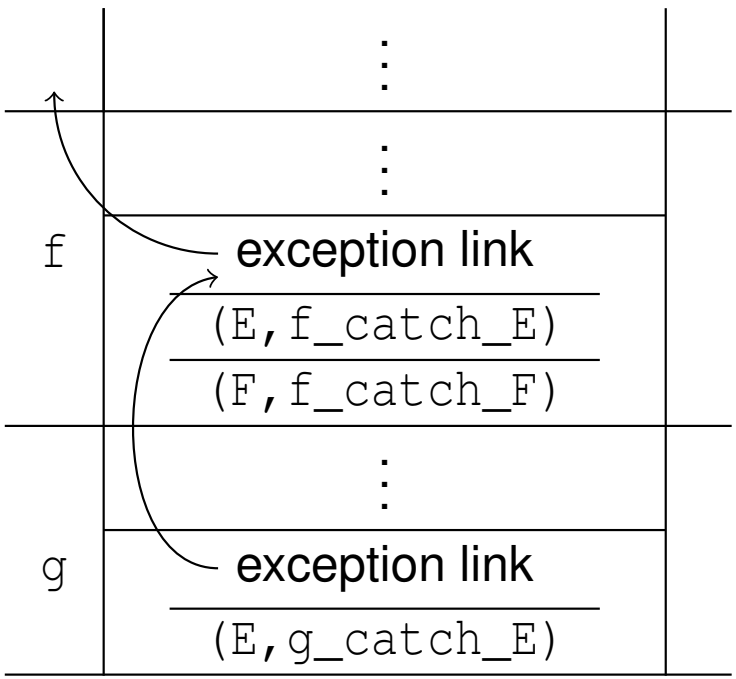
Example

```
void f() { try {  g()  } catch(E e) { } catch(F f) { } }
void g() { try {  ...  } catch(E e) { } }
```



Example

```
void f() { try {  g()  } catch(E e) { } catch(F f) { } }
void g() { try {  ...  } catch(E e) { } }
```



How can one find the right handler *without* searching the stack of exception handlers installed at run time? In particular, how can one find `f_catch_F` when `g()` throws an exception of type `F`?

Static List of Exception Handlers

All exception handlers of a program are listed in one table E of tupels $(\tau, \alpha, \beta, \varepsilon)$, where

- τ is the type of the handled exception,
- α is the address of the handler,
- $[\beta, \varepsilon]$ is the range of instruction addresses of the `try` block.

When an exception of type t occurs at an address x, the list of exception handlers is searched:

- Let H be the set of all matching handlers from E, i.e., $H := \{h \mid h = (\tau, \alpha, \beta, \varepsilon) \in E, \beta \leq x \leq \varepsilon, t \leq \tau\}$.
- When $H \neq \varnothing$, then $\min_\tau\left(\max_\beta(H)\right)$ is the desired handler (innermost `try` block due to $\max_\beta$ and most specific type due to $\min_\tau$).
- When $H = \varnothing$ and x lies in the `main` method of the program, there is no exception handler.
- Otherwise, let x' be the address from which the method containing x has been called (i.e., the call site of the current method). Repeat the search with $x := x'$.

Comparison of Exception Handling Methods

runtime costs	dynamic	static
entry in `try` block	naïve $O(c)$ optimized $O(1)$	none
exit from `try` blocks	$O(1)$	none
finding a handler	$O(k)$	naïve $O(d\,n)$ optimized (cf. Tutorial)
jump to handler	$O(1)$	$O(1)$

static parameters:

- n: total number of exception handlers in the program

dynamic parameters:

- k: number of active `catch` blocks at the time of the exception
- c: number of `catch` blocks of the current `try` block
- d: depth of the call stack

Overview

1. C Runtime System
   - C Runtime System
   - C Standard Library
   - Processes and Libraries
2. Error Handling in C
   - `errno`
   - `longjmp`
   - Signals
3. Application of `longjmp` and Signals
   - Thread Scheduling
   - Exceptions
4. Interactions with Other Runtime Systems
   - Interactions

Interactions

Due to the close connection between operating system and the C runtime system (at least in Unix-like operating systems), runtime systems and virtual machines for other/higher languages are often based on the C runtime system.

This in not unproblematic. Although runtime systems of higher languages try to hide the C runtime system from the programs of the higher languages, programs can still call functions from the C runtime system. Unintended interactions can arise between

- the C runtime system and some functions of the runtime system of the higher language,
- calls to the C runtime system from the runtime system of the higher language and the program itself.

Interactions with the Runtime System

Problematic interactions can occurs, among others, at

- memory management,
- threads,
- input/output,
- exceptions.

Interactions: Memory Management

`malloc` and other memory management functions for C are based on the (not standardized any more) functions `sbrk` und `brk`.

Program Break

Using the function `void *sbrk(intptr_t increment)` and `int brk(void *addr)` one can modify the *program break*, i.e., the upper bound of the heap of a process. The program break is the address of the first byte past the end of the heap.

`sbrk(x)` increments the program break by `x` and returns the previous program break. By calling `sbrk(0)`, one can find the current program break.

Interactions: Memory Management

`malloc` and other memory management functions for C are based on the (not standardized any more) functions `sbrk` und `brk`.

Program Break

Using the function `void *sbrk(intptr_t increment)` and `int brk(void *addr)` one can modify the *program break*, i.e., the upper bound of the heap of a process. The program break is the address of the first byte past the end of the heap.

`sbrk(x)` increments the program break by `x` and returns the previous program break. By calling `sbrk(0)`, one can find the current program break.

(Note: modern implementations of the C runtime system prefer `mmap` to allocate memory in the mmap area over modifying the program break; both approaches can be used on current Unix-like systems.)

Memory Management

Memory management using `malloc` is not always the best strategy for a higher language. Compiler and runtime system for higher languages often have more precise information about the sizes, life times and locality ($\rightsquigarrow$ cache) of allocated memory areas.

Memory Management

Memory management using `malloc` is not always the best strategy for a higher language. Compiler and runtime system for higher languages often have more precise information about the sizes, life times and locality ($\rightsquigarrow$ cache) of allocated memory areas.

Therefore, higher languages tend to implement their own memory management (without relying on `malloc`). But: there is no coexistence of `malloc` with explicit use of (`s`)`brk`. Possible solutions:

1. The higher language provides specialized routines for `malloc`, `free`, etc. for the C runtime system (which are compatible with the higher languages memory management).
2. Memory management is performed using standard `malloc`. The heap of the higher language is a big allocation made with `malloc`; the higher language manages its heap inside this allocated area.

The second alternative is more portable but less flexible.

Threads

To implement threads, there are two principal possibilities:

Threads

- green threads: runtime system implements the context switch
- native threads: threads of the higher language are implemented using operating system threads

Green threads are simpler to implement and do not require proper concurrency in the runtime system. On the other hand, they cannot use the concurrency provided by multicore processors. Formerly, green threads were more common, today native threads are the common case.

And in the future?

Threads

To implement threads, there are two principal possibilities:

Threads

- green threads: runtime system implements the context switch
- native threads: threads of the higher language are implemented using operating system threads

Green threads are simpler to implement and do not require proper concurrency in the runtime system. On the other hand, they cannot use the concurrency provided by multicore processors. Formerly, green threads were more common, today native threads are the common case.

And in the future? Both! Manycore architectures require that a program provides much more parallelism than today, but on a system with 4 cores one does not want to start 10.000 native threads.

Example for Green+Native Threads: Fibonacci in GpH

```
fib :: Int -> Int
fib n  =  if  n < 2  then  n  else  fib (n-1) + fib (n-2)

fibPar :: Int -> Int
fibPar n = if n < 2 then n
           else x `par`            -- evaluate x in new spark
                (y `pseq` x+y)     -- when y is ready, compute x+y
    where
      -- restrict parallelism:
      -- use sequential implementation for small n
      f = if n > 20 then fibPar else fib
      x = f (n-1)
      y = f (n-2)

main :: IO ()
main = print (fibPar 42)
```

`par` starts the evaluation of its left argument in a kind of green thread (called "spark" in Glasgow Parallel Haskell). When a native thread is available, the next unevaluated spark is evaluated.

Statistics of the runtime system for `./Fib.par +RTS -s -N4` (`-N4`: use 4 native threads in the computation):

```
MUT time (elapsed)       GC time  (elapsed)
Task  0 (worker) :    0.00s    (  0.08s)       0.00s    (  0.00s)
Task  1 (worker) :    1.31s    (  1.31s)       0.00s    (  0.00s)
Task  2 (worker) :    1.21s    (  1.31s)       0.02s    (  0.00s)
Task  3 (worker) :    0.00s    (  1.31s)       0.00s    (  0.00s)
Task  4 (worker) :    1.31s    (  1.31s)       0.00s    (  0.00s)
Task  5 (worker) :    1.30s    (  1.31s)       0.00s    (  0.00s)

SPARKS: 185470 (41 converted, 162047 pruned)

INIT time 0.00s ( 0.00s elapsed) MUT time 5.19s ( 1.31s elapsed) GC
time 0.02s ( 0.00s elapsed) EXIT time 0.00s ( 0.00s elapsed) Total time
5.21s ( 1.31s elapsed)
```

The program generated 185470 sparks. 41 sparks were mapped to native threads (i.e., a parallel computation was started), 162047 have been eliminated by the runtime system (to save the overhead of context switches).

Green Threads and I/O

Green threads are well-suited for asynchronous tasks that do not required real parallel computation. Example: A webserver starts a green thread for every HTTP connection. Every thread has to wait for the request (HTTP header) to arrive and sends the requested data back over the network.

Green Threads and I/O

Green threads are well-suited for asynchronous tasks that do not required real parallel computation. Example: A webserver starts a green thread for every HTTP connection. Every thread has to wait for the request (HTTP header) to arrive and sends the requested data back over the network.

Green threads are susceptible to block the whole runtime system when making blocking calls to the operating system.

Example: A thread calls `write` to send data to the network. The call to `write` blocks because the send buffer is full. Consequence: the whole process (including all green threads) “hangs” because the operating system is not aware of the existence of other green threads.

Non-Blocking I/O

To solve the problem with blocking all green threads, a runtime system has to use *non-blocking* I/O.

To this end, one has to use the proper asynchronous interface for I/O (e.g., `aio_read` etc.) or one calls `read`, `write`, etc. only when data is available or there is space in the buffer, respectively:

- install a signal handler for `SIGPOLL`,
- switch file handles to non-blocking mode,
- enable signaling of “data available” or “space in buffer available”, respectively, for file handles,
- schedule other green threads when not all data can be read or written at once and continue I/O operation when `SIGPOLL` arrives.

In earlier times (and in some situations today) this was the standard on Unix-like operating systems before native threads existed.

Exceptions

Exceptions cause problems in two areas:

- There can only be a single handler for a given signal $\rightarrow$ a library in C and the high-level language runtime system may both want to install a handler for, e.g., `SIGFPE`.
- A `longjmp` in the error handling on the C side can confuse the state (stack, etc.) of the high-level language runtime system.

Example for Interactions with Exceptions

1. The high-level language runtime system installs a handler for `SIGFPE`.
2. The high-level language calls a library that itself installs another handler for `SIGFPE` which uses `longjmp`.
3. The high-level language calls a function from the library and there a floating point exception occurs.

When the handler of the library is used, `longjmp` can cause unforeseeable effects on the high-level language. When 1 and 2 are swapped, the handler of the high-level language's runtime system is called without the execution being in the high-level language part of the program.

This interaction *cannot* be solved cleanly.

Example from Practice: GNU MP

The runtime system of the Glasgow Haskell Compiler (GHC) uses the GNU Multiple Precision (GMP) library to implement integers with arbitrary size.

In a project at the former Char of Programming (Prof. Lengauer) a library is called from Haskell that itself uses GMP to compute with arbitrary sized integers.

Consequence: segfaults, not always immediatly, sometimes in the C code, sometimes in Haskell code.

How long do you thing the lecturer had to look for the cause of the problem (and what is the problem)?

Example from Practice: GNU MP

The runtime system of the Glasgow Haskell Compiler (GHC) uses the GNU Multiple Precision (GMP) library to implement integers with arbitrary size.

In a project at the former Char of Programming (Prof. Lengauer) a library is called from Haskell that itself uses GMP to compute with arbitrary sized integers.

Consequence: segfaults, not always immediatly, sometimes in the C code, sometimes in Haskell code.

How long do you thing the lecturer had to look for the cause of the problem (and what is the problem)? Fortunately, not very long, because other people hat already done a (long) search for the cause of the problem and the lecturer only needed to do an Internet search ;-)

Memory Management of GNU MP

GMP tries to do the right thing and offers the user a way to determine how GMP allocates memory. To this end, one can use the function

```
void mp_set_memory_functions( void *(*alloc_func_ptr) (size_t),
void *(*realloc_func_ptr) (void *, size_t, size_t),
void (*free_func_ptr) (void *, size_t));
```

to tell GMP which functions to use for (re-)allocation and freeing of memory.

The Haskell runtime system sets these functions to its memory management routines that allocate from Haskell's garbage collected heap. When one calls GMP functions from C code, then memory alloctions (in GMP) are still made on the Haskell heap without the Haskell side being ready to handle these.

Solution: on the C side, set allocation routines of GMP to `malloc`, etc. to make GMP use the C heap and before returning to Haskell restore the routines that Haskell installed in GMP.

Summary of Interactions

The interactions between the C runtime system and other runtime systems are a mine field and not all problems have satisfactory solutions.

What is (from the point of view of this lecture) the underlying weakness of the C runtime system?

Summary of Interactions

The interactions between the C runtime system and other runtime systems are a mine field and not all problems have satisfactory solutions.

What is (from the point of view of this lecture) the underlying weakness of the C runtime system?

Services such as memory allocation, signals, exceptions are not *virtualized*; therefore, different “guests” easily get in each other’s ways.

Consequence: Design Recommendation for Libraries

How can one avoid (further) interactions by design in libraries?

Consequence: Design Recommendation for Libraries

How can one avoid (further) interactions by design in libraries? Explicit virtualization of the library by providing a context parameter:

- Every user of the library has to allocate a *context* before using the library.
- All functions of the library work “inside” (or: relative to) a context.
- The context virtualizes the library by encapsulating the state of the library w.r.t. memory management, error states, etc.

The library can be used by several users (“guests”) without different users interacting unintendedly.

Example: Communicator argument in the *Message Passing Interface* (MPI).

Another Solution (for Some Casse)

To avoid inadvertent interactions, there exists another possibility: use the virtualization of the operating system.

Another Solution (for Some Casse)

To avoid inadvertent interactions, there exists another possibility: use the virtualization of the operating system.

In principle, the runtime system of a high-level language can create one (or several) operating system processes for calling C functions (and external libraries not designed for several concurrent users). These separate processes have their own memory management, signals, etc.

Complication: parameters have to be passed between processes, i.e., across separate address spaces.